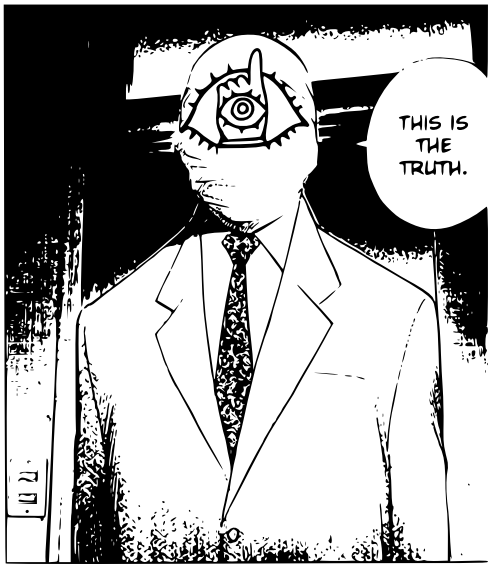
Marco Casu

SAPIENZA
UNIVERSITÀ DI ROMA

Faculty of Information Engineering, Computer Science and Statistics
Department of Computer, Control and Management Engineering
Master's degree in Artificial Intelligence and Robotics

This document summarizes and presents the topics for the Robotics 2 course for the Master's degree in Artificial Intelligence and Robotics at Sapienza University of Rome. The document is free for any use. If the reader notices any typos, they are kindly requested to report them to the author.



CONTENTS

1 Introduction **3**
1.1 An Overview in Dynamics 3
1.2 Planning vs Feedback Control 7

2 Advanced Topics on Kinematics **9**
2.1 Calibration 9
2.2 Kinematic Redundancy 12
2.2.1 Local Resolution Methods 13

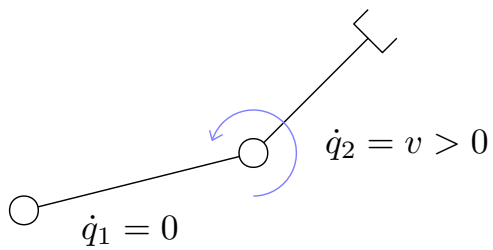
CHAPTER

1

INTRODUCTION

1.1 An Overview in Dynamics

Previously, in the Robotics 1 course, we described the kinematic behavior of a robotic manipulator, assuming that no force are involved, and that we can control directly the angular velocity of each joint, without any kind of resistance. We consider the following example

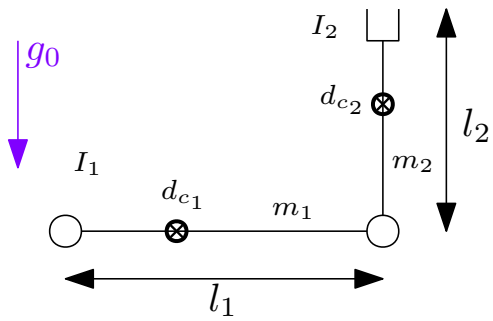


it is clear that, considering the positive angular velocity $v > 0$, in the next configuration the second link will rotate counter clock wise, while the first joint remain fixed. If we have to take in account the dynamics, we should also consider

- the center of mass of the first and the second link
- the mass of the first and the second link
- the inertia the first and the second link

and instead of describing the control trough angular velocities on the two joints, we define the torques τ_1, τ_2 applied on them. In this case (we ignore the effects of gravity), by applying a zero torque $\tau_1 = 0$ on the first joint, and a positive torque $\tau_2 > 0$ on the second joint, both joints will move, according with the laws of rigid body dynamics.

Let's consider the following example of a 2R planar robot, with the joint axis orthogonal respect to the direction of gravity.



where I_1, I_2 are the moment of inertia of the links around their center of mass, d_{c_1}, d_{c_2} is the distance along the links between the joint and the center of mass, m_1, m_2 are the mass of the links, and l_1, l_2 the lengths. The initial configuration at $t = 0$ is

$$\mathbf{q}_0 = \begin{pmatrix} 0 \\ \frac{\pi}{2} \end{pmatrix} \qquad \dot{\mathbf{q}}_0 = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \tag{1.1}$$

Consequently, also $\dot{\mathbf{p}} = \mathbf{0}$ (the velocity of the effector end). We want to know, what will be $\ddot{\mathbf{q}}$ (and consequently $\ddot{\mathbf{p}}$) in the initial configuration at $t = 0$, by considering the dynamics and the forces. The Jacobian matrix of this simple manipulator is

$$J = \begin{pmatrix} -l_1 s_1 - l_2 s_{12} & -l_2 s_{12} \\ l_1 c_1 + l_2 c_{12} & l_2 c_{12} \end{pmatrix} \tag{1.2}$$

in the initial configuration $\mathbf{q}(0) = \mathbf{q}_0$ we have

$$J(\mathbf{q}_0) = \begin{pmatrix} -l_2 & -l_2 \\ l_1 & 0 \end{pmatrix} \tag{1.3}$$

from the differential relation

$$\ddot{\mathbf{p}} = J\ddot{\mathbf{q}} + \dot{J}\dot{\mathbf{q}} \tag{1.4}$$

since $\dot{\mathbf{q}}_0 = \mathbf{0}$

$$\ddot{\mathbf{p}}_0 = \begin{pmatrix} -l_2 & -l_2 \\ l_1 & 0 \end{pmatrix} \ddot{\mathbf{q}}_0 \tag{1.5}$$

we know that, by finding the initial joint acceleration $\ddot{\mathbf{q}}_0$ we can predict the next configuration and where the end effector will move. The full dynamic model is a system of differential equation and is the following

$$M(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{c}(\mathbf{q}, \dot{\mathbf{q}}) + \mathbf{g}(\mathbf{q}) = \mathbf{0} \tag{1.6}$$

where M is the 2×2 inertia matrix, and depends only on the current configuration, $\mathbf{c}$ is a term representing the centrifugal and coriolis forces, and depends also on the velocities $\dot{\mathbf{q}}$. The last term $\mathbf{g}$ is the gravity term, and depends on the current configuration. We are not explaining in details how the model is derived, since this is an introductory section, and all these topics will be described and expanded in the following chapters.

Since the robot is at rest, the $\mathbf{c}$ vector is null, so

$$\ddot{\mathbf{q}} = M^{-1}(\mathbf{q})\mathbf{g}(\mathbf{q}) \tag{1.7}$$

the inertia matrix in the initial configuration (that for now we get for granted) is

$$M(\mathbf{q}_0) = \begin{pmatrix} I_1 + m_1 d_{c_1}^2 + I_2 + m_2 d_{c_2}^2 + m_2 l_1^2 & I_2 + m_2 d_{c_2}^2 \\ I_2 + m_2 d_{c_2}^2 & I_2 + m_2 d_{c_2}^2 \end{pmatrix} \tag{1.8}$$

Theorem 1 (Steiner) *If I is the moment of inertia around an axis of a rigid body, and d is the distance between this axis and the center of mass, the inertia around the parallel axis passing trough the center of mass will be $\bar{I} = I + md^2$, where m is the mass of the body.*

In the inertia matrix we see the terms $I_i + m_i d_{c_i}^2$, that is exactly the moment of inertia around the parallel axis passing trough the center of mass, that we denote $\bar{I}_i$.

$$M(\mathbf{q}_0) = \begin{pmatrix} \bar{I}_1 + \bar{I}_2 + m_2 l_1^2 & \bar{I}_2 \\ \bar{I}_2 & \bar{I}_2 \end{pmatrix} \tag{1.9}$$

This inertia matrix is *always* positive definite, so it is invertible, in particular

$$M^{-1}(\mathbf{q}_0) = \frac{1}{\det M(\mathbf{q}_0)} \begin{pmatrix} \bar{I}_2 & -\bar{I}_2 \\ -\bar{I}_2 & \bar{I}_1 + \bar{I}_2 + m_2 l_1^2 \end{pmatrix} = \tag{1.10}$$

$$\frac{1}{(\bar{I}_1 + m_2 l_1^2)\bar{I}_2} \begin{pmatrix} \bar{I}_2 & -\bar{I}_2 \\ -\bar{I}_2 & \bar{I}_1 + \bar{I}_2 + m_2 l_1^2 \end{pmatrix} \tag{1.11}$$

the gravity term is the following

$$\mathbf{g}(\mathbf{q}_0) = \begin{pmatrix} m_1 d_{c_1} g_0 + m_2 l_1 g_0 \\ 0 \end{pmatrix} = \begin{pmatrix} (m_1 d_{c_1} + m_2 l_1) g_0 \\ 0 \end{pmatrix} \tag{1.12}$$

the first component $m_1 d_{c_1} g_0 + m_2 l_1 g_0$ represents the total gravitational torque acting on the first joint.

- $m_1 d_{c_1} g_0$: This is the torque generated by the weight of the first link.
- $m_2 l_1 g_0$: This is the torque generated by the weight of the second link acting at the end of the first link.

due to the vertical alignment of the initial configuration, the gravity does not exerts any torque on the second joint. By putting together the equations we get:

$$\ddot{\mathbf{q}}_0 = \frac{(m_1 d_{c_1} + m_2 l_1) g_0}{(\bar{I}_1 + m_2 l_1^2)\bar{I}_2} \begin{pmatrix} -\bar{I}_2 \\ \bar{I}_2 \end{pmatrix} = \begin{pmatrix} \star < 0 \\ \star > 0 \end{pmatrix} \tag{1.13}$$

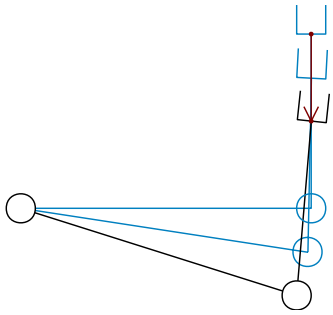
so the first joint will rotate clockwise while the second counter clockwise. We want to know if the end effector will fall to the left or to the right. We compute

$$\ddot{\mathbf{p}}_0 = \begin{pmatrix} -l_2 & -l_2 \\ l_1 & 0 \end{pmatrix} \ddot{\mathbf{q}}_0 = \tag{1.14}$$

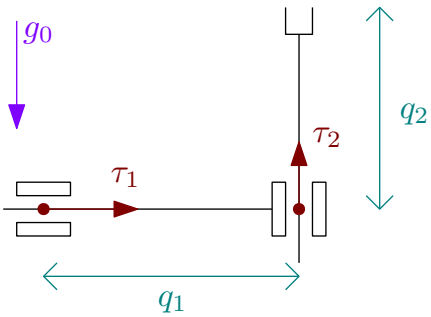
$$\ddot{\mathbf{p}}_0 = \begin{pmatrix} -l_2 & -l_2 \\ l_1 & 0 \end{pmatrix} \frac{(m_1 d_{c_1} + m_2 l_1) g_0}{(\bar{I}_1 + m_2 l_1^2)\bar{I}_2} \begin{pmatrix} -\bar{I}_2 \\ \bar{I}_2 \end{pmatrix} = \tag{1.15}$$

$$\ddot{\mathbf{p}}_0 = \begin{pmatrix} 0 \\ -\frac{l_1 (m_1 d_{c_1} + m_2 l_1) g_0}{\bar{I}_1 + m_2 l_1^2} \end{pmatrix} \tag{1.16}$$

so the end effector will accelerato downward without moving on the x axis.



Now, we consider a simpler example from a dynamic point of view, but we will show the process to derive the dynamic model. We consider a planar 2P robot, where the joint's are orthogonal, in this simple case the Jacobian matrix is the identity. For this manipulator there will be no angular terms, since all the joint's are prismatic, we still refers with torques to the linear forces τ_1, τ_2 applied on the prismatic joints.



We consider two different methods to derive the dynamic model (both these methods we will further explained in details in the their relative sections).

Newton-Euler

This methods perform a balance of forces acting on the system of rigid bodies. We start by the last link, backtracking towards the first. The forces acting on the second link are the torque τ_2 (the commands) and the gravity, so the dynamic equation for the second link (according to the Newton laws) is:

$$\tau_2 - m_2 g_0 = m_2 \ddot{q}_2 \tag{1.17}$$

on the first link, there is no force exerts from the gravity (since this link is constrained to move only along the x axis), so is affected only by the command torque τ_1 and the inertia of the second link (which opposes its motion):

$$\tau_1 - m_2 \ddot{q}_1 = m_1 \ddot{q}_1 \tag{1.18}$$

by putting the two equations together we obtain the following system

$$\begin{pmatrix} m_1 m_2 & 0 \\ 0 & m_2 \end{pmatrix} \begin{pmatrix} \ddot{q}_1 \\ \ddot{q}_2 \end{pmatrix} + \begin{pmatrix} 0 \\ m_2 g_0 \end{pmatrix} = \begin{pmatrix} \tau_1 \\ \tau_2 \end{pmatrix} \tag{1.19}$$

the first matrix M is the inertia matrix while the vector $\begin{pmatrix} 0 \\ m_2 g_0 \end{pmatrix}$ is the gravity term.

Euler-Lagrange

This methods is based on the energy of the system. The kinetic energy T is given by the sum of the kinetic energies of the two links $T_1 + T_2$, the same applies for the potential energy $U = U_1 + U_2$ (that in this case, is given by the gravity field). The kinetic energy of the first link (that can move only along the x axis) is

$$T_1 = \frac{1}{2} m_1 \dot{q}_1^2 \tag{1.20}$$

since the second link can move in both directions, and their velocity is given by $\mathbf{v}_{c_2} = \begin{pmatrix} \dot{q}_1 \\ \dot{q}_2 \end{pmatrix}$, the kinetic energy is

$$T_2 = \frac{1}{2} m_2 \|\mathbf{v}_{c_2}\|^2 = \frac{1}{2} m_2 (\dot{q}_1^2 + \dot{q}_2^2) \tag{1.21}$$

the potential energy of the first link is constant, since it can't move along the y axis, we denote this constant energy just U_1. The potential energy of the second link, given by the gravity field, is $U_2 = m_2 g_0 q_2$.

The *Lagrangian* is a function describing the dynamics of the system (will be explained in the next chapters), is given by the difference between the kinetic and the potential energy

$$\mathcal{L} = T - U \tag{1.22}$$

depends on the current configuration $\mathbf{q}$ and the velocity $\dot{\mathbf{q}}$. In this case we have

$$\mathcal{L}(\mathbf{q}, \dot{\mathbf{q}}) = \frac{1}{2} m_1 \dot{q}_1^2 + \frac{1}{2} m_2 (\dot{q}_1^2 + \dot{q}_2^2) - U_1 - m_2 g_0 q_2 \tag{1.23}$$

there is a simple principle in Physics, the trajectories of the system (the evolution over time of $\mathbf{q}$ and $\dot{\mathbf{q}}$) are the one that *minimizes the action over time*, that is the following scalar function

$$\mathcal{S} = \int_0^t \mathcal{L}(\mathbf{q}, \dot{\mathbf{q}}) dt \tag{1.24}$$

this is a problem of Calculus of Variations (the minimization of a functional over a set of functions) and is proved to be solved by the trajectories that satisfies the **Euler-Lagrange Equations**:

$$\frac{d}{dt} \frac{\partial \mathcal{L}}{\partial \dot{q}_i} - \frac{\partial \mathcal{L}}{\partial q_i} = \tau_i, \quad 1 \leq i \leq n \tag{1.25}$$

for our 2P robotic system, we have the two following equations

$$\begin{cases} (m_1 + m_2) \ddot{q}_1 = \tau_1 \\ m_2 \ddot{q}_2 + m_2 g_0 = \tau_2 \end{cases} \tag{1.26}$$

That fully describes the model.

1.2 Planning vs Feedback Control

We review some concepts already explored in the course of Robotics 1 about the differences between planning and feedback control. We recall that a good controller in every robotics application, should incorporate the offline planning procedure with an online closed loop system.

We take as an example a simple body with mass m that lies on a plane with zero friction. This body can move only along the direction parallel to the plane, and it's position is defined by the variable x, and it is applied an horizontal force F, the equation that describes the dynamics of the object is of course $F = m\ddot{x}$.

We want to plan a motion, starting from the initial condition

$$\begin{pmatrix} x \\ \dot{x} \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \tag{1.27}$$

and reaching the final condition

$$\begin{pmatrix} x \\ \dot{x} \end{pmatrix} = \begin{pmatrix} x_d \\ 0 \end{pmatrix} \tag{1.28}$$

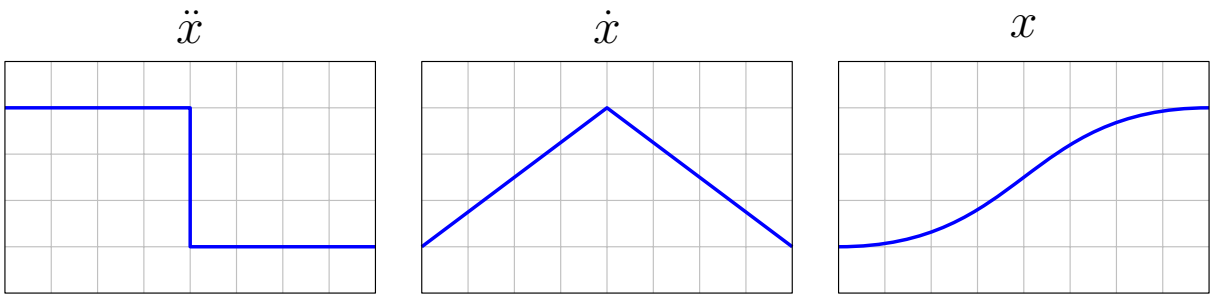
We can plan the motion by integrating over time $\ddot{x} = \frac{F}{m} = u$. We already seen the simple bang-bang profile, in that case, the displacement in position reached in T seconds by applying a command input of amplitude u is

$$x = \frac{T^2}{4} u \tag{1.29}$$

if u is limited by F_{max} the minimum time to reach $x = x_d$ starting from $x = 0$ is

$$T = \sqrt{\frac{4mx_d}{F_{max}}} \tag{1.30}$$

where $F_{max} = mu_{max}$.



This approach execute the desired motion in nominal condition in the minimum time, but it requires some information about the dynamical model (in this case, the value of m). Of course, in real situations, there are always some disturbance forces that we can't control or predict, so this approach will eventually leads to a configuration that is not exactly the desired one, in some cases, with an high error.

The feedback approach consists in using the state variables in the commands, it doesn't necessary requires the dynamic model, but some sensors to measure the position, velocity, and other quantities. In this simple example, the general approach is the following

$$m\ddot{x} = u(x, \dot{x}) \tag{1.31}$$

where u is the control rule. We consider the following PD law:

$$m\ddot{x} = k_P(x_d - x) - k_D\dot{x} \tag{1.32}$$

the system is still linear. We track the error and it's derivatives

$$\begin{pmatrix} e = x_d - x \\ \dot{e} = -\dot{x} \\ \ddot{e} = -\ddot{x} \end{pmatrix} \tag{1.33}$$

We express the dynamics in term of the errors and we get the second order equation:

$$m\ddot{e} + k_P e + k_D\dot{e} = 0 \tag{1.34}$$

this homogeneous equation can be solved analytically by knowing m, k_P, k_D and the initial condition. The value of $e(t)$ converges to zero if m, k_D and k_P have the same sign, since m is the mass (and is always positive), it is sufficient to choose positive values for k_P, k_D. This law works even if there are some disturbance, we can combine the feed forward action with the feedback loop by choosing a force command

$$F = F_f + k_P(x_f - x) + k_D(\dot{x}_f - \dot{x}) \tag{1.35}$$

where $F_f(t)$ realizes the desired trajectory in nominal condition and $x_f(t)$ is the evolution over time of x in the nominal condition.

This simple PD control law doesn't work in situations where the system is subject to undamped persistent forces. Let's consider a body of mass m subject to the gravity, that can move only along the axis orthogonal to the ground. We call y the position of the body, and we can apply a force F as the command input. The dynamic model is described by the equation

$$m\ddot{y} = F - mg_0. \tag{1.36}$$

If we choose a PD control law for F the dynamic equation is

$$m\ddot{y} = k_P(y_d - y) - k_D\dot{y} - mg_0. \tag{1.37}$$

where y_d is the target position and 0 is the target velocity. The error is $e = y_d - y$, the temporal evolution of the error is described by the equation

$$k_P e + k_D\dot{e} + m\ddot{e} - mg_0 = 0 \tag{1.38}$$

Will $e(t)$ converge to zero? Let's decide some values for the data just to give an example, let $m = k_P = k_D = 1$.

$$e + \dot{e} + \ddot{e} - 9.8 = 0 \tag{1.39}$$

if the initial conditions are $e(0) = y_d,\ \dot{e}(0) = 0$ the solution is

$$e(t) = \exp\left(-\frac{1}{2}t\right)\left[(y_d - 9.8)\cos\left(\frac{\sqrt{3}}{2}t\right) + \frac{y_d - 9.8}{\sqrt{3}}\sin\left(\frac{\sqrt{3}}{2}t\right)\right] + 9.8 \tag{1.40}$$

since $\lim_{t\to\infty}\exp(-\frac{1}{2}t) = 0$ we have

$$\lim_{t\to\infty} e(t) = 9.8 \neq 0 \tag{1.41}$$

so the desired state will be never reached. It is sufficient to add an integral term in the control law by making it a PID controller:

$$k_I \int e d\tau \tag{1.42}$$

the final dynamic equation with the inclusion of this term is

$$k_P e + k_D\dot{e} + k_I \int_0^t e d\tau + m\ddot{e} - mg_0 = 0 \tag{1.43}$$

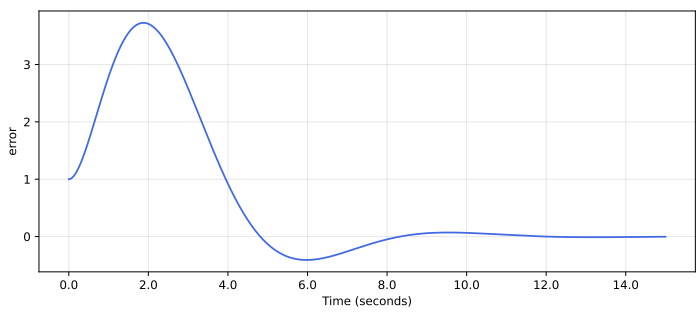


Figure 1.1: Discrete time simulation with $m = 1, y_d = 1, k_P = k_D = 2, k_I = 1, \Delta t = 0.0001$.

CHAPTER

2

ADVANCED TOPICS ON KINEMATICS

2.1 Calibration

A measurement system have an high repeatability if different measurements of the same quantity provides similar results, and is accurate if the measured quantity is near to the ground truth. A robot can be seen as a measurement system, the DH parameters are used in the direct kinematics to compute the task vector from a given configuration. If the predicted position of the end effector is far from the real position, the parameters are wrong, there is a lack in accuracy and the **calibration procedure** is needed.

For simplicity, we consider a manipulator with n (all revolut) joints. The DH parameters are

$$\boldsymbol{\alpha} = \begin{pmatrix} \alpha_1 \\ \vdots \\ \alpha_n \end{pmatrix}, \quad \mathbf{a} = \begin{pmatrix} a_1 \\ \vdots \\ a_n \end{pmatrix}, \quad \mathbf{d} = \begin{pmatrix} d_1 \\ \vdots \\ d_n \end{pmatrix} \tag{2.1}$$

from these parameters we can compute the *nominal* direct kinematics

$$\mathbf{y}_{e,nom} = \mathbf{k}_e(\boldsymbol{\alpha}, \mathbf{a}, \mathbf{d}, \boldsymbol{\theta}) \tag{2.2}$$

$\boldsymbol{\theta}$ is the set of joint angles measured by the encoders. The calibration procedure is essential to have a perfect direct kinematics, especially in the industrial context.

- due to the tolerances in the mechanical construction and in assembly of the links and the joints, there is a small errors in the actual end effector pose respect to the one given by the direct kinematics.
- the errors distributed along the are are amplified, due due to the open kinematic chain structure of most robots
- the goal of the calibration procedure is to recover as much as possible this end effector position error by adjusting the DH parameters.

The calibration procedure needs a sampling operations, where the robot is set in different configuration, when an external device measure the real end effector pose, in such way we can compare this ground truth with the pose predicted by the direct kinematics.

Let $\mathbf{y}_e$ to be the real end effector pose measured by the external system, and $\mathbf{y}_{e,nom}$ the one given by the direct kinematics, we consider the error

$$\Delta\mathbf{y} = \mathbf{y}_e - \mathbf{y}_{e,nom} \tag{2.3}$$

from the first order Taylor expansion of the direct kinematics, we can write the following linear relation

$$\Delta\mathbf{y} = \frac{\partial \mathbf{k}_e}{\partial \boldsymbol{\alpha}}\Delta\boldsymbol{\alpha} + \frac{\partial \mathbf{k}_e}{\partial \mathbf{a}}\Delta\mathbf{a} + \frac{\partial \mathbf{k}_e}{\partial \mathbf{d}}\Delta\mathbf{d} + \frac{\partial \mathbf{k}_e}{\partial \boldsymbol{\theta}}\Delta\boldsymbol{\theta} \tag{2.4}$$

The small vectors $\Delta\boldsymbol{\alpha}, \Delta\mathbf{a}, \Delta\mathbf{d}, \Delta\boldsymbol{\theta}$ are the small errors in the DH parameters. The relation can be rewritten in matrix form

$$\Delta\mathbf{y}_e = \Phi\Delta\boldsymbol{\varphi} \tag{2.5}$$

where

$$\Phi = \begin{pmatrix} \frac{\partial\mathbf{k}_e}{\partial\boldsymbol{\alpha}} & \frac{\partial\mathbf{k}_e}{\partial\mathbf{a}} & \frac{\partial\mathbf{k}_e}{\partial\mathbf{d}} & \frac{\partial\mathbf{k}_e}{\partial\boldsymbol{\theta}} \end{pmatrix} \in Mat(6\times 4n) \tag{2.6}$$

$$\Delta\boldsymbol{\varphi} = \begin{pmatrix} \Delta\boldsymbol{\alpha} \\ \Delta\mathbf{a} \\ \Delta\mathbf{d} \\ \Delta\boldsymbol{\theta} \end{pmatrix} \in \mathbb{R}^{4n} \tag{2.7}$$

notice how $\Delta\mathbf{y}$ is given by the measurement, Φ is evaluated with the nominal parameters and $\Delta\boldsymbol{\varphi}$ is the unknown error in the DH parameters.

For example, with a 2R planar robot, where the direct kinematics is

$$\mathbf{k}(\mathbf{a},\boldsymbol{\theta}) = \begin{pmatrix} a_1\cos\theta_1 + a_2\cos(\theta_1+\theta_2) \\ a_1\sin\theta_1 + a_2\sin(\theta_1+\theta_2) \end{pmatrix} \tag{2.8}$$

if the we assume that the parameters $\alpha, \mathbf{d}$ are correct, the Φ matrix to consider is

$$\Phi = \begin{pmatrix} \frac{\partial\mathbf{k}}{\partial\mathbf{a}} & \frac{\partial\mathbf{k}}{\partial\boldsymbol{\theta}} \end{pmatrix} = \begin{pmatrix} c_1 & c_{12} & -a_1s_2 - a_2s_{12} & -a_2s_{12} \\ s_1 & s_{12} & a_1c_1 + a_2c_{12} & a_2c_{12} \end{pmatrix} \tag{2.9}$$

Instead of considering one simple point to measure the error, one can sample $l >> n$ points, to get l matrices Φ_i and l cartesian errors $\Delta\mathbf{y}_{e,i}$. In this way we can construct the matrix

$$\overline{\Phi} = \begin{pmatrix} \Phi_1 \\ \vdots \\ \Phi_l \end{pmatrix} \in Mat(6l\times 4n) \tag{2.10}$$

and the vector

$$\overline{\Delta\mathbf{y}}_e = \begin{pmatrix} \Delta\mathbf{y}_{e,1} \\ \vdots \\ \Delta\mathbf{y}_{e,l} \end{pmatrix} \in \mathbb{R}^{6l} \tag{2.11}$$

the system of equation to solve in order to find the errors in the DH parameters is

$$\overline{\Phi}\Delta\boldsymbol{\varphi} = \overline{\Delta\mathbf{y}}_e \tag{2.12}$$

$\overline{\Phi}$ is called **regressor matrix**. The system in (2.12) is an overdetermined system of equations, and can be solved by considering the pseudo inverse of $\overline{\Phi}$

$$\Delta\boldsymbol{\varphi} = \overline{\Phi}^{\#}\overline{\Delta\mathbf{y}}_e \tag{2.13}$$

$$\overline{\Phi}^{\#} = (\Phi^T\Phi)^{-1}\Phi^T \tag{2.14}$$

Instead of finding the pseudo inverse, an iterative algorithm can be applied to find $\Delta\boldsymbol{\varphi}$ that considers sequentially the measurements.

The following table report the cartesian errors resulting from a calibration procedure made on a ABB IRB 120 6R industrial robot on 1000 random configurations, 50 were used in the algorithm to correct the DH parameters, and the other 950 were used for the validation.

	before calibration	after calibration
average	1.745 mm	0.193 mm
median	1.567 mm	0.180 mm
standard deviation	1.043 mm	0.085 mm
min	0.050 mm	0.010 mm
max	4.423 mm	0.516 mm

Algorithm 1 Calibration Algorithm

$\epsilon_r = 10^{-4}$ //tolerance for convergence
$i_{max} = 50$ //maximum iterations
$\boldsymbol{\varphi}^{(0)} = \boldsymbol{\varphi}_{nom}$ //nominal DH parameters
$\overline{\Phi}^{(0)} = \overline{\Phi}(\boldsymbol{\varphi}^{(0)})$
$\overline{\Delta\mathbf{y}}_e^{(0)} = \overline{\Delta\mathbf{y}}_e(\boldsymbol{\varphi}^{(0)})$
$i = 0$
while $i \leq i_{max}$ or $\|\overline{\Delta\mathbf{y}}_e^{(i)}\| > \epsilon_r$ **do**
  $\Delta\boldsymbol{\varphi}^{(i)} = \left(\overline{\Phi}^{(i)^T}\overline{\Phi}^{(i)}\right)^{-1}\overline{\Phi}^{(i)^T}\overline{\Delta\mathbf{y}}_e^{(i)}$ //use of the pseudo inverse $\overline{\Phi}^{(i)\#}$
  $\boldsymbol{\varphi}^{(i+1)} = \boldsymbol{\varphi}^{(i)} + \Delta\boldsymbol{\varphi}^{(i)}$ //update of the DH parameters
  $\overline{\Phi}^{(i+1)} = \overline{\Phi}(\boldsymbol{\varphi}^{(i+1)})$ //recompute regressor matrix
  $\overline{\Delta\mathbf{y}}_e^{(i+1)} = \overline{\Delta\mathbf{y}}_e(\boldsymbol{\varphi}^{(i+1)})$ //recompute cartesian error
end while
if $\|\overline{\Delta\mathbf{y}}_e^{(i)}\| \leq \epsilon_r$ **then**
  return $\boldsymbol{\varphi}^{(i)}$
end if
return no solution

Let's consider the following example, there is a manipulator with 2 joints, the *ground truth* values for the parameters $\mathbf{a}$ is

$$\mathbf{a} = \begin{pmatrix} a_1 \\ a_2 \end{pmatrix} = \begin{pmatrix} 100 \\ 1 \end{pmatrix} \tag{2.15}$$

the nominal values given in the DH table are

$$\bar{\mathbf{a}} = \begin{pmatrix} \bar{a}_1 \\ \bar{a}_2 \end{pmatrix} = \begin{pmatrix} 101 \\ 2 \end{pmatrix} \tag{2.16}$$

there are 4 measurements of the cartesian position, the regressor matrix $\overline{\Phi}$ for these is

$$\overline{\Phi} = \begin{pmatrix} -1 & 1 \\ 0 & 1 \\ 1 & 0 \\ 2 & -1 \end{pmatrix} \tag{2.17}$$

the cartesian displacement measured is

$$\overline{\Delta\mathbf{y}_e} = \begin{pmatrix} 0 \\ -1 \\ -1 \\ -1 \end{pmatrix} \tag{2.18}$$

we have to solve the system

$$\begin{pmatrix} -1 & 1 \\ 0 & 1 \\ 1 & 0 \\ 2 & -1 \end{pmatrix} \Delta\mathbf{a} = \begin{pmatrix} 0 \\ -1 \\ -1 \\ -1 \end{pmatrix} \tag{2.19}$$

can be solved by considering the pseudo inverse of $\overline{\Phi}$:

$$\Delta\mathbf{a} = \begin{pmatrix} 0 & 1/3 & 1/3 & 1/3 \\ 1/3 & 2/3 & 1/3 & 0 \end{pmatrix} \begin{pmatrix} 0 \\ -1 \\ -1 \\ -1 \end{pmatrix} = \begin{pmatrix} -1 \\ -1 \end{pmatrix} \tag{2.20}$$

we update the parameters

$$\bar{\mathbf{a}}_{calibrated} = \bar{\mathbf{a}} + \Delta\mathbf{a} = \begin{pmatrix} 100 \\ 1 \end{pmatrix}. \tag{2.21}$$

2.2 Kinematic Redundancy

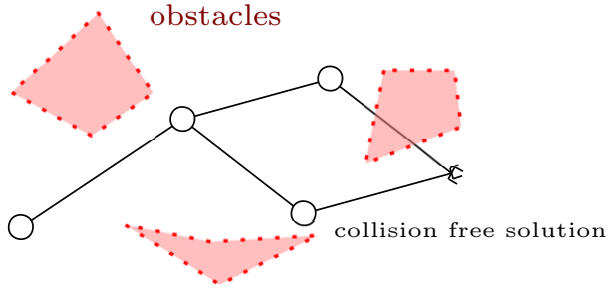
The direct kinematics $\mathbf{y} = \mathbf{k}(\mathbf{q})$ is the map between the task space and the configuration space

$$\mathbf{k} : \mathcal{C} \to \mathcal{T} \tag{2.22}$$

we usually denote $n = \dim \mathcal{C}$ the dimension of the configuration space and $m = \dim \mathcal{T}$ the dimension of the task space. A robot is **kinematically redundant** if $n > m$ (there are more degrees of freedom than strictly needed for executing the task). The redundancy of a robot is a relative concept, doesn't depend on the number of joints, but on the number of dimension of the task space, a 6R robot is not redundant for the task of positioning and orientating the end effector, but is redundant for the task of just positioning.

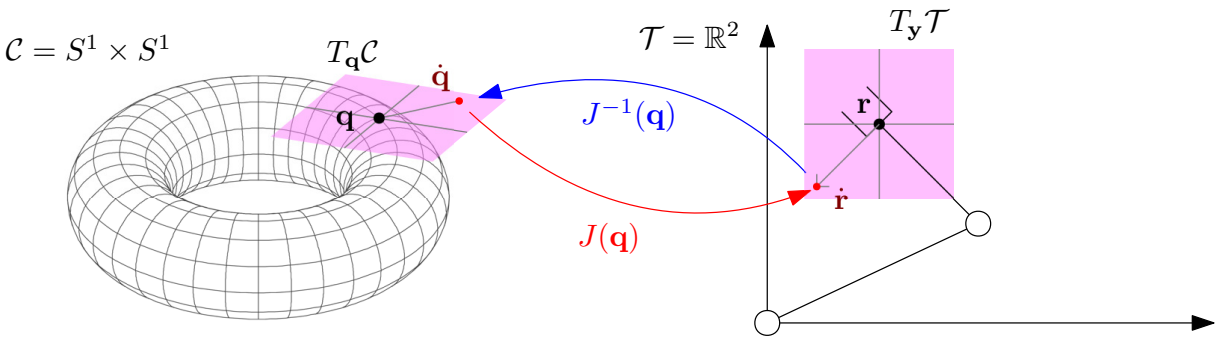
A redundant robot have an infinite number of solution for the inverse kinematics in the regular case, the redundancy can be used in various ways, for example

- avoid collision with obstacles in the cartesian space
- avoid kinematic singularities in the joint space
- increase manipulability in specified directions
- uniformly distribute/limit joint velocities and/or accelerations
- optimize energy consumption or needed motion torques
- ...



We define a **self motion** of a redundant robot, a motion in the joint space (arm reconfiguration) that doesn't change the task vector $\mathbf{y}$. In a given configuration, one may perform a self motion to get to a configuration that improve/optimize in some way the behavior of the robotic system.

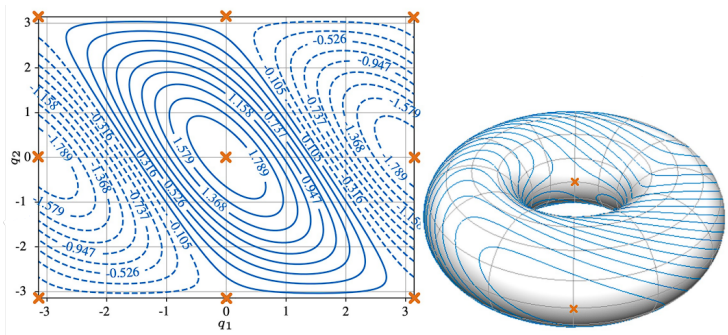
The solution to the inverse kinematics to perform a self motion are mainly sought at differential level, by considering the Jacobian matrix, due to the linearity, we recall that the Jacobian is the linear mapping between the tangent spaces of $\mathcal{C}$ and $\mathcal{T}$.



In practice, the configuration that are in the self motion on a given position $\mathbf{p}$ satisfies

$$\mathbf{q} \in \mathcal{C} \ : \ \mathbf{k}(\mathbf{q}) = \mathbf{p} \tag{2.23}$$

the following image represents the points on the configuration space of a planar 2R that are mapped on the same task vector.



Without entering in the details (for now), the *general redundancy resolution* is an optimization problem that can be solved offline (globally) or online (locally). In both methods, the objective function is a general function of the state space $H(\mathbf{q}, \dot{\mathbf{q}})$, and the goal is to find a trajectory for $\mathbf{q}(t)$ and $\dot{\mathbf{q}}(t)$, thus, redundancy resolution is a calculus of variations problem.

- **Global methods**: Given a desired trajectory $\mathbf{y}(t)$ and an initial configuration $\mathbf{q}(t_0)$, to find an evolution over $[t_0, t_0 + T]$, is involved a functional of the type

$$\int_{t_0}^{t_0+T} H(\mathbf{q}, \dot{\mathbf{q}})dt \tag{2.24}$$

  that have to be minimized (or maximized) over $\mathbf{q}(t), t \in [t_0, t_0 + T]$. This problem is usually too difficult to solve.

- **Local methods**: Usually at each time step $t = kT_s$ (T_s is the sampling time) given a task velocity $\dot{\mathbf{y}}$ to realize we solve the problem of finding the joint velocity $\dot{\mathbf{q}}$ that realizes such $\dot{\mathbf{y}}$ while optimizing a given function $H(\mathbf{q}, \dot{\mathbf{q}})$. Then the discrete step of update is

$$\mathbf{q}((k+1)T_s) = \mathbf{q}(kT_s) + T_s\dot{\mathbf{q}}(kT_s) \tag{2.25}$$

  this is more easy to solve and usually requires the resolution of a linear quadratic problem.

2.2.1 Local Resolution Methods

There are three classes of methods to solve $J\dot{\mathbf{q}} = \dot{\mathbf{y}}$:

- Jacobian-based: Among the infinite solutions, the one that minimizes a suitable norm is chosen
- null-space methods: a term is added to the previous solution so as not to affect execution of the task trajectory, i.e., belonging to the null-space $\mathcal{N}(J)$
- task augmentation: redundancy is reduced/eliminated by adding $s \leq n - m$ further auxiliary tasks to make the problem "squared".

Jacobian Based

We have to solve the redundant problem from a differential level

$$J\dot{\mathbf{q}} = \dot{\mathbf{y}} \tag{2.26}$$

where J have m rows and n columns, with $n > m$. We will consider solutions of the form

$$\dot{\mathbf{q}} = K\dot{\mathbf{y}} \tag{2.27}$$

where K is a matrix with n rows and m columns. We want K to satisfy the following condition

$$JKJ = J \tag{2.28}$$

such a matrix is called a **generalized inverse** of J (depends on the current configuration $\mathbf{q}$), there are infinite matrix that satisfies that condition. A generalized inverse can realize the desired task velocity $\dot{\mathbf{y}}$ if it is in the range of J.

$$\forall \dot{\mathbf{y}} \in \mathcal{R}(J) \ : \ J(K\dot{\mathbf{y}}) = JKJ\dot{\mathbf{q}} = J\dot{\mathbf{q}} = \dot{\mathbf{y}} \tag{2.29}$$

An example of a generalized inverse is the *right inverse*, let J to be a Jacobian of the form

$$J = \begin{pmatrix} J_a & J_b \end{pmatrix} \tag{2.30}$$

where $\det J_a \neq 0$, then, the right inverse is

$$K_r = \begin{pmatrix} J_a^{-1} \\ 0 \end{pmatrix} \tag{2.31}$$

this is equivalent to "freezing" some degrees of freedom.

Another example of a generalized inverse is the **pseudo inverse matrix**, that is the matrix $J^{\#}$ that satisfies the four following properties:

$$JJ^{\#}J = J \tag{2.32}$$

$$J^{\#}JJ^{\#} = J^{\#} \tag{2.33}$$

$$(JJ^{\#})^T = JJ^{\#} \tag{2.34}$$

$$(J^{\#}J)^T = J^{\#}J \tag{2.35}$$

if the matrix J is full rank, and $n > m$, then the pseudo inverse can be computed in closed form:

$$J^{\#} = J^T(JJ^T)^{-1}. \tag{2.36}$$

Example: if

$$J = \begin{pmatrix} 1 & 2 & 3 \end{pmatrix}$$

the pseudo inverse is

$$\begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix} \left(\begin{pmatrix} 1 & 2 & 3 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix} \right)^{-1} = \frac{1}{14} \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}.$$

Example: Now we consider a square singular matrix. First of all, let J_ϵ to be the following parametrized matrix

$$J_\epsilon = \begin{pmatrix} 3 & 0 \\ 0 & \epsilon \end{pmatrix}$$

we know that the inverse of this matrix is

$$J_\epsilon^{-1} = \begin{pmatrix} \frac{1}{3} & 0 \\ 0 & \frac{1}{\epsilon} \end{pmatrix}$$

with $\epsilon \to 0$ (the matrix becomes singular), the right-bottom element of the inverse $\frac{1}{\epsilon}$ goes to infinity. So the matrix

$$J = \begin{pmatrix} 3 & 0 \\ 0 & 0 \end{pmatrix}$$

isn't invertible and the pseudo inverse just replace the "infinite-becoming" term with 0:

$$J^{\#} = \begin{pmatrix} \frac{1}{3} & 0 \\ 0 & 0 \end{pmatrix}.$$

Example: Let's consider the following redundant full rank matrix

$$J = \begin{pmatrix} 1 & 0 & 2 \\ 2 & 1 & -1 \end{pmatrix}$$

we consider a generalized inverse, in particular, the right inverse:

$$J = \begin{pmatrix} \begin{pmatrix} 1 & 0 \\ 2 & 1 \end{pmatrix} & \begin{pmatrix} 2 \\ -1 \end{pmatrix} \end{pmatrix}$$

$$K_r = \begin{pmatrix} \begin{pmatrix} 1 & 0 \\ 2 & 1 \end{pmatrix}^{-1} \\ \begin{pmatrix} 0 & 0 \end{pmatrix} \end{pmatrix}$$

$$K_r = \begin{pmatrix} 1 & 0 \\ -2 & 1 \\ 0 & 0 \end{pmatrix}.$$

We already seen in the Robotics 1 course that the pseudo inverse realizes the desired task velocity by choosing the joint's velocity $\dot{\mathbf{q}}$ of minimum norm. In particular, given the problem $J\dot{\mathbf{q}} = \dot{\mathbf{y}}$, even if $\dot{\mathbf{y}}$ is not in the range space of J, the pseudo inverse finds the minimum norm solution among the $\dot{\mathbf{q}}$ that minimizes

$$\|\dot{\mathbf{y}} - J\dot{\mathbf{q}}\|^2. \tag{2.37}$$

If J is singular and JJ^T cannot be inverted, the formula $J^\# = J^T(JJ^T)^{-1}$ can't be used, and to find $J^\#$ we need numerical methods that relies on the *singular value decomposition* of J.

Definition 1 (SDV) *For any matrix J, there exists a decomposition*

$$J = U\Sigma V^T \tag{2.38}$$

where U and V are orthonormal matrices, and Σ is a diagonal matrix.

This decomposition have special properties, the columns of U are the eigen vectors of JJ^T, and the columns of V are the eigen vectors of J^TJ. The matrix Σ have the following form:

$$\Sigma = \begin{pmatrix} \sigma_1 & & & \\ & \ddots & & 0_{m\times(n-m)} \\ & & \sigma_m & \end{pmatrix} \tag{2.39}$$

it follows that

$$\sigma_1 \geq \sigma_2 \cdots \geq \sigma_\rho > 0 \tag{2.40}$$
$$\sigma_{\rho+1} = \sigma_{\rho+2} \cdots = \sigma_m = 0 \tag{2.41}$$

where ρ is the rank of J. The numbers $\sigma_1 \ldots, \sigma_\rho$ are the **singular values** of J. The eigen vectors of JJ^T (columns of U) are associated to the eigen values $\sigma_1^2 \ldots, \sigma_\rho^2$. The last $n-\rho$ columns of V are a basis for the null space of J.

$$J\mathbf{v}_i = \sigma_i \mathbf{u}_i, \quad (i = 1 \ldots, \rho) \tag{2.42}$$
$$J\mathbf{v}_i = 0, \quad (i = \rho + 1 \ldots n) \tag{2.43}$$

we denote $\mathbf{u}_i$ the i-th column vector of U and $\mathbf{v}_i$ the i-th column vector of V.

We want to show that a vector $\mathbf{u}_i$ is an eigen vector of JJ^T. Since $J = U\Sigma V^T$ we have

$$JJ^T = U\Sigma V^T V \Sigma^T U^T \tag{2.44}$$

since V is orthonormal, we have that $V^T = V^{-1}$ so $V^TV = I$:

$$JJ^T = U\Sigma\Sigma^T U^T \tag{2.45}$$